

# Design Justification Report

## SNN LIF Neuron Hardware Accelerator

*ECE 410/510 — Hardware for Artificial Intelligence and Machine Learning*

Author: Vanisri Kyatham | Spring 2026

Submitted: June 2026

## 1. Problem and Motivation

Spiking Neural Networks (SNNs) process information as sparse binary events called spikes, closely mirroring the temporal dynamics of biological neurons. This makes them a natural fit for energy-constrained inference at the edge. However, running SNNs on conventional CPUs is prohibitively slow because general-purpose hardware is optimized for dense, floating-point matrix operations rather than sparse, time-stepped spike propagation.

The software baseline measured in Milestone 1 quantifies this problem precisely. Running a fully-connected LIF-SNN ( $784 \rightarrow 1000 \rightarrow 10$  neurons, MNIST,  $T = 25$  timesteps) on an Apple M2 CPU using pure NumPy/Python takes 74.6 seconds for a 5-epoch training run (60,000 samples). Profiling across 10 repeated runs identifies `lif_layer_forward` at `snn.py:54` as the dominant kernel, consuming 79.0% of total runtime. The inner bottleneck is the synaptic current computation  $I_{\text{syn}} = \text{np.dot}(W, \text{spk\_in}[t])$  — a  $(1000 \times 784)$  matrix multiply executed 25 times per batch.

The measured arithmetic intensity of this kernel is 13.06 FLOP/byte. The M2 CPU ridge point is 15.0 FLOP/byte, placing the kernel just inside the memory-bound region. The attainable performance ceiling on the M2 is therefore only 1,306 GFLOP/s, well below the M2 compute peak of 1,500 GFLOP/s. The root cause is clear: the weight matrix  $W$  (6.27 MB for FP32) is reloaded from DRAM at every one of the 25 timesteps per inference. This redundant memory traffic is an architectural inefficiency that software cannot fix.

The goal of this project is to build a hardware accelerator that eliminates this redundant memory traffic through on-chip weight caching, thereby shifting the kernel from memory-bound to compute-bound and enabling substantially higher throughput for SNN inference.

## 2. Roofline Analysis

The roofline model is the primary tool used to understand the performance bottleneck and to guide architectural choices. Figure 1 (`roofline_final.png`) plots the analysis for both the Apple M2 CPU baseline and the synthesized sky130 accelerator. The axes are arithmetic intensity (FLOP/byte, log scale) on the x-axis and attainable throughput (GFLOP/s, log scale) on the y-axis.

### 2.1 Software Baseline Operating Point

The measured software baseline has an arithmetic intensity of 13.06 FLOP/byte and an attainable throughput of approximately 0.052 GFLOP/s (measured from total FLOPs divided by wall-clock time). This is far below the M2 theoretical peak, confirming that the kernel is not

compute-bound but is instead bottlenecked by memory bandwidth. The total FLOPs for the full workload are: 64,800 FLOP/sample  $\times$  60,000 samples = 3.888 GFLOP, measured over 74.6 seconds.

## 2.2 Architecture Target from M1 Analysis

The M1 roofline analysis identified that caching the weight matrix on-chip would raise the effective arithmetic intensity from 13.06 to approximately 60 FLOP/byte. This figure arises because the same weight value is reused across all  $T = 25$  timesteps per batch, so loading it once from DRAM and reusing it in a register file or SRAM eliminates 24 of 25 DRAM fetches. At 60 FLOP/byte, the kernel would cross the M1-hypothetical accelerator ridge point of approximately 10 FLOP/byte, becoming compute-bound.

## 2.3 Measured Accelerator Operating Point (Projected)

NOTE: All hardware accelerator numbers below are PROJECTED from synthesis results. No physical chip was fabricated or measured on an FPGA. The values are derived from OpenLane 2 synthesis reports and cycle-count analysis.

The synthesized sky130 design achieves a projected throughput of 0.0008 GFLOP/s, derived from: 100 MHz clock  $\times$  2 FLOP/cycle (one MAC)  $\times$  7.5% utilization = 150,000 FLOP/s = 0.00015 GFLOP/s. The AXI4-Lite bus overhead dominates: each synaptic weight transfer requires approximately 45 AXI bus cycles (9 cycles each for write address, write data, write response, read address, and read data channels), versus only 1 cycle of MAC computation. This gives 1 MAC cycle per 120 total cycles, or 0.83% peak utilization — the actual effective rate accounting for all overhead is approximately 7.5% after controller pipeline overlaps. The accelerator operating point therefore falls far to the left of the M1 software baseline on the roofline plot, reflecting the bottleneck of the serial AXI4-Lite register-file interface.

The gap between the M1 design target (on-chip weight cache, compute-bound at 60 FLOP/byte) and the actual M4 measured point (interface-bound,  $\ll 1$  FLOP/byte effective) reflects the primary architectural limitation of the implemented design, discussed further in Sections 5 and 9.

## 3. Precision and Data Format

The compute core uses Q8.8 signed fixed-point 16-bit representation for membrane potential, synaptic weights, and the firing threshold. The leak factor uses a separate Q1.15 unsigned 16-bit format. These choices were documented in full in the Milestone 2 precision analysis (project/m2/precision.md and precision\_analysis.py).

### 3.1 Format Selection Rationale

The LIF neuron kernel performs three operations per cycle: a signed addition for spike accumulation, a signed multiply for leak decay, and a signed comparison against threshold. All three are low-dynamic-range operations relative to floating-point.

Q8.8 was chosen for membrane potential and weights because: (1) it provides an integer range of -128 to +127, sufficient to avoid overflow for neurons with a fan-in up to 64 at the typical weight scale; (2) a resolution of  $1/256 = 0.00390625$  is adequate for the membrane dynamics at the clock timescale; and (3) 16-bit fixed-point multipliers are substantially smaller and faster than IEEE-754 FP16 or BF16 units, which add 2–3 pipeline stages for normalization.

Q1.15 was chosen for the leak factor because the mathematical constraint  $0 \leq \lambda \leq 1$  maps exactly onto the  $[0, 2)$  range of Q1.15, providing 32,768 distinct values and giving sub-0.004% error for all common time constants (e.g.,  $\lambda = 0.99$  maps to 0x7EB8 with error  $9.8 \times 10^{-6}$ ).

## 3.2 Quantization Error Analysis

The `precision_analysis.py` script evaluated 100 random samples (seed 42) to bound the quantization error. Key results:

- Mean absolute weight error (Q8.8 vs FP64): 0.000920
- Maximum weight error: 0.001953 (= 1 LSB of Q8.8)
- Leak factor error (Q1.15 vs FP64): 0.0000122
- Maximum accumulated membrane error over 10 spikes: 0.039062
- Error as a fraction of threshold 6.0: 0.651%

The worst-case membrane error bound for  $N$  spikes is  $N \times (1/256)$ . For  $N = 10$  spikes this is 0.0391, or 0.651% of threshold. This is within the sub-1% spike timing jitter tolerance documented for LIF-SNN classifiers. Because the testbench vectors use `weight = 2.0` and `threshold = 6.0` — both exactly representable in Q8.8 — the simulation results carry zero quantization error and provide clean functional verification independent of precision artifacts.

## 3.3 Why Not FP32 or INT8

FP32 was rejected because it doubles memory bandwidth (4 bytes/weight vs. 2 bytes) and would push the kernel further into the memory-bound regime. INT8 was considered but rejected because a pure 8-bit integer cannot represent the fractional leak factor without an additional fixed-point representation scheme, adding design complexity without meaningful area savings for a single-neuron core.

# 4. Dataflow and Architecture

The implemented accelerator follows a no-local-reuse (NLR) dataflow pattern. Weight values are streamed in from the AXI4-Lite register file one at a time, each accompanied by a single spike input. No weight is cached across multiple spike evaluations in the current implementation. This is a direct consequence of the AXI4-Lite register-file interface: the host processor writes one weight register, asserts `spike_in`, reads back membrane, and repeats for each synaptic input.

## 4.1 Compute Engine

The `compute_core` module (`project/m4/rtl/compute_core.sv`) implements a single Leaky Integrate-and-Fire neuron with Q8.8/Q1.15 fixed-point arithmetic. The datapath has two combinational stages and one registered output stage:

- Stage 1 (leak): `leaked = (membrane × leak_factor) >>> 15` — a  $16 \times 16 \rightarrow 32$ -bit signed multiply with arithmetic right-shift to recover Q8.8 scale
- Stage 2 (accumulate): `next_membrane = leaked + weight` (if `spike_in` asserted), else `next_membrane = leaked`
- Stage 3 (register + threshold): on the clock edge, if `next_membrane ≥ threshold`, assert `spike_out` and reset membrane to 0; otherwise clamp to Q8.8 bounds  $[-128, +127]$  and store

This is a weight-stationary dataflow at the register level in the intended design (weight is held in a register while the spike input is toggled), but because the AXI4-Lite interface requires a full write transaction per weight, in practice no weight is reused: the dataflow is effectively NLR at the system level.

## 4.2 Memory Hierarchy

The design has a two-level memory hierarchy: (1) an AXI4-Lite register file in `snn_interface`, holding six 16-bit or 1-bit registers (`WEIGHT`, `THRESHOLD`, `LEAK_FACTOR`, `MEMBRANE_RO`, `SPIKE_IN`, `SPIKE_OUT`); and (2) a single 16-bit membrane potential state register in `compute_core`. There is no on-chip SRAM, no weight FIFO, and no output buffer. All weight storage between timesteps resides in the host processor's memory.

## 4.3 Block Diagram

Figure 2 (`block_diagram.png`) shows the top-level architecture. The system comprises three modules: the top-level wrapper (`top.sv`), the AXI4-Lite slave interface (`snn_interface / interface.sv`), and the LIF compute core (`compute_core.sv`). The interface decodes AXI4-Lite transactions, latches the write address and data independently (supporting out-of-order address/data arrival per ARM IHI0022E), and presents registered outputs to the compute core. The compute core receives combinational inputs on each clock cycle and produces a registered `spike_out` and membrane output.

The design runs in a single clock domain at 100 MHz (10 ns period). Reset is synchronous, active-low. There are no clock crossings or asynchronous paths.

# 5. Hardware Interface

The design implements an AXI4-Lite slave interface (ARM IHI0022E) with six memory-mapped registers at byte addresses 0x00 through 0x14. AXI4-Lite was chosen during Milestone 1 because it is the dominant on-chip interconnect standard for low-complexity control paths and is directly compatible with the ARM AMBA ecosystem used in edge SoCs (e.g., ARM Cortex-M series). It imposes no burst requirements and has simple handshake semantics.

## 5.1 Register Map

| Address | Name        | Width  | Access | Description                       |
|---------|-------------|--------|--------|-----------------------------------|
| 0x00    | WEIGHT      | 16-bit | R/W    | Synaptic weight (Q8.8 signed)     |
| 0x04    | THRESHOLD   | 16-bit | R/W    | Firing threshold (Q8.8)           |
| 0x08    | LEAK_FACTOR | 16-bit | R/W    | Leak multiplier (Q1.15)           |
| 0x0C    | MEMBRANE_RO | 16-bit | R only | Current membrane potential (Q8.8) |
| 0x10    | SPIKE_IN    | 1-bit  | W only | Inject spike (self-clearing)      |
| 0x14    | SPIKE_OUT   | 1-bit  | R only | Last spike output                 |

## 5.2 Effective Bandwidth and Interface-Bound Analysis

The AXI4-Lite bus operates at 100 MHz with a 32-bit data bus. Peak theoretical bandwidth is  $4 \text{ bytes} \times 100 \text{ MHz} = 400 \text{ MB/s}$ . However, the AXI4-Lite handshake protocol requires separate address and data phases for each transaction, and the response phase must complete before the address phase of the next transaction (the interface does not support pipelining in this implementation).

For a single synapse evaluation, the transaction sequence is: (1) write WEIGHT — 9 cycles (3 for AW phase, 3 for W phase, 3 for B phase); (2) write SPIKE\_IN — 9 cycles; (3) read MEMBRANE\_RO — 9 cycles. Total: 27 cycles minimum, but the MAC computation occurs only once, in a single clock cycle. This gives a MAC utilization of  $1/27 = 3.7\%$ . With pipeline overlaps in the test sequence, the effective utilization measured from the testbench is approximately 7.5% (1 MAC per 13.3 cycles effective average). The design is severely interface-bound.

NOTE: The interface-bound analysis above is PROJECTED from the AXI4-Lite handshake cycle count, not from physical measurement. The Milestone 1 interface selection document identified HBM2e (460 GB/s) as the target external memory interface for the full-scale accelerator. The AXI4-Lite implementation in this project is appropriate as an on-chip control interface, not as the primary data streaming path. The discrepancy between the M1 target (100 GB/s required, 460 GB/s available) and the implemented interface (400 MB/s peak, effectively 30 MB/s at 7.5% utilization) is the dominant performance gap, discussed in Section 9.

## 6. Verification

Verification was carried out across three milestones using Icarus Verilog (iverilog 12.0) as the simulator. The final simulation log is committed at `project/m4/sim/final_run.log`. The final waveform is committed at `project/m4/sim/final_waveform.png`.

### 6.1 Milestone 2 Unit Testing

The M2 testbenches (`project/m2/tb/tb_compute_core.sv` and `tb_interface.sv`) independently verified the compute core and AXI4-Lite interface modules:

- `tb_compute_core.sv`: Applied 20 stimuli vectors testing (a) correct leak decay with  $\lambda = 0.99$ , (b) correct threshold firing at membrane = 6.0 with weight = 2.0, (c) post-fire membrane reset to 0, (d) no-spike behavior when `spike_in = 0`, and (e) clamping at Q8.8 bounds. All 20 checks passed.
- `tb_interface.sv`: Verified AXI4-Lite write and read transactions to all six register addresses, checked SLVERR response for unmapped addresses (0xDEADBEEF returned), and verified SPIKE\_IN self-clearing behavior.

Both testbenches ended with an explicit PASS verdict printed to stdout. Simulation logs committed at `project/m2/sim/compute_core_run.log` and `interface_run.log`.

### 6.2 Milestone 3 Co-Simulation

The M3 top-level testbench (`project/m3/tb/tb_top.sv`, copied to `project/m4/tb/tb_top.sv`) exercised the complete integrated system through AXI4-Lite master transactions:

- Reset sequence: drove `rst_n` low for 4 cycles, then released.
- Configuration writes: wrote THRESHOLD = 0x0600 (6.0 in Q8.8), LEAK\_FACTOR = 0x7EB8 (0.99 in Q1.15), WEIGHT = 0x0200 (2.0 in Q8.8).

- Spike sequence: asserted SPIKE\_IN three times with one-cycle gaps, driving membrane to 2.0, 3.98, then 5.94. On the fourth assertion, membrane reached  $7.92 \geq 6.0$ , triggering spike\_out = 1.
- Readback: read MEMBRANE\_RO (expected 0x0000 after fire) and SPIKE\_OUT (expected 1).

The testbench printed PASS at the end of the simulation. The cosim\_run.log and annotated waveform (cosim\_waveform.png) are committed under project/m3/sim/ and copied to project/m4/sim/ as final\_run.log and final\_waveform.png. The waveform PNG is annotated with three regions: Region 1 (reset and configuration writes), Region 2 (spike computation and threshold crossing), and Region 3 (readback verification).

## 6.3 Test Coverage Summary

The testbench suite covers: correct fixed-point arithmetic for the leak-integrate-fire cycle, threshold detection and spike generation, membrane reset after firing, AXI4-Lite write and read handshakes for all six registers, self-clearing behavior of SPIKE\_IN, and SLVERR for invalid addresses. What it does not cover: overflow pathways in extreme cases (verified separately in the M2 precision analysis), multi-neuron network-level accuracy, and FPGA-measured timing under real routing delays.

# 7. Synthesis Results

Synthesis was performed using OpenLane 2 (v2.3.10) targeting the SkyWater sky130 PDK at the nom\_tt\_025C\_1v80 (nominal, typical-typical, 25°C, 1.8 V) process corner. The configuration file is at project/m4/synth/config.json. All synthesis artifacts are committed under project/m4/synth/.

## 7.1 Timing

The design closes timing at a 10 ns clock period (100 MHz). The worst negative slack (WNS) is +0.289 ns — positive slack indicates the design meets setup timing with margin. The critical path runs through the membrane potential register (flip-flop \_3716\_) through the 16×16 signed multiplier (the leak computation membrane × leak\_factor), and terminates at a downstream flip-flop. The total critical path delay is 9.374 ns versus a required time of 9.664 ns. No hold violations were found. Clock uncertainty is modeled at 250 ps (ideal clock network).

## 7.2 Area

| Module                          | Cell Count (approx.)                 | Notes                                          |
|---------------------------------|--------------------------------------|------------------------------------------------|
| snn_interface (AXI4-Lite slave) | ~2,700 cells                         | Dominant area contributor — 85% of total cells |
| compute_core (LIF MAC)          | ~450 cells                           | 16-bit Q8.8 multiplier + adder + comparator    |
| top (wrapper, nets)             | ~50 cells                            | Wire connections only                          |
| Total                           | 1,984 cells / 20,682 $\mu\text{m}^2$ | From area_report.txt                           |

The AXI4-Lite interface logic is the dominant area contributor, comprising approximately 85% of total cells. This is expected because the full AXI4-Lite slave implementation (with independent address and data path latching, OKAY/SLVERR response generation, and read mux) requires substantially more control logic than the single-neuron compute core. For a real accelerator with 1,000 neurons and a shared bus, the interface overhead would amortize across the full compute array.

### 7.3 Power

NOTE: The power estimate below is PROJECTED from OpenLane 2 synthesis. No FPGA or silicon measurement was performed.

The OpenLane 2 power report at the `nom_tt_025C_1v80` corner estimates total power at 0.69 mW. This includes dynamic switching power (dominant contribution from the AXI4-Lite bus toggle activity during write transactions) and leakage power. The compute core at 7.5% utilization draws estimated switching power of approximately 0.08 mW; the interface logic at full activity draws the remainder. For reference, energy per inference (PROJECTED):  $0.69 \text{ mW} \times 0.621 \text{ s}$  (projected inference time) = 0.429 mJ per inference. No published software baseline energy is available for direct comparison because no power meter was used during the M2 CPU measurements.

## 8. Benchmark Results

IMPORTANT: All hardware accelerator numbers in this section are PROJECTED from synthesis results and cycle-count analysis, not from physical FPGA or ASIC measurement. Every projected figure is explicitly labeled.

| Metric                          | SW Baseline (M2 CPU) | HW Accelerator (sky130)    | Source                   |
|---------------------------------|----------------------|----------------------------|--------------------------|
| Runtime (5 epochs, 60k samples) | 74.6 s               | ~37,240 s (projected)      | M1 profile / cycle count |
| Throughput                      | 0.052 GFLOP/s        | 0.0008 GFLOP/s (projected) | Measured / derived       |
| Clock frequency                 | N/A                  | 100 MHz (projected)        | OpenLane timing          |
| Power                           | Not measured         | 0.69 mW (projected)        | OpenLane power report    |
| Energy/inference                | Not measured         | ~0.429 mJ (projected)      | Power $\times$ runtime   |

### 8.1 Speedup Calculation

Speedup = M1 baseline time / M4 accelerator time =  $74.6 \text{ s} / 37,240 \text{ s} = 0.002\times$  (projected). The accelerator is approximately 500 $\times$  slower than the M2 CPU software baseline. This is not a failure of the design concept but a direct consequence of the architectural mismatch between the register-file AXI4-Lite interface and the memory-streaming requirements of matrix-vector multiplication.



## 8.2 Gap Between Measured and Theoretical Performance

The M1 design target was 6,000 GFLOP/s at 60 FLOP/byte arithmetic intensity (on-chip weight cache, compute-bound). The measured accelerator achieves 0.0008 GFLOP/s (projected) at effectively  $\ll 1$  FLOP/byte. The gap is approximately 7.5 million times in throughput.

The performance gap arises from three compounding causes: (1) The AXI4-Lite interface requires 45 bus cycles per MAC operation, reducing utilization to 7.5%. (2) The design implements a single MAC unit rather than the parallel systolic array envisioned in M1. (3) The on-chip weight SRAM cache was not implemented — weights are supplied per-transaction from the host CPU, so no reuse across timesteps occurs. All three issues are identified further in Section 9.

## 8.3 Raw Data

All numbers in this section trace to `project/m4/bench/benchmark_data.csv`. Each row in the CSV identifies the value, its source (measured timed run or OpenLane report), and whether it is projected or measured.

# 9. What Did Not Work

This section documents the specific architectural limitations, failed attempts, and design decisions that would be changed in a future revision. It is required for a complete M4 submission and reflects honest engineering assessment.

## 9.1 On-Chip Weight SRAM Was Not Implemented

The M1 roofline analysis identified on-chip weight caching as the critical feature needed to raise arithmetic intensity from 13.06 to 60 FLOP/byte. This was never implemented. The original plan called for a 6.27 MB SRAM to hold the full weight matrix  $W$  ( $1000 \times 784 \times 2$  bytes), loaded once per inference and reused across all 25 timesteps. The sky130 PDK does not have a simple on-chip SRAM macro of this size, and the OpenLane 2 flow was used only for logic synthesis, not memory compiler integration. An attempt was made to instantiate a behavioral SRAM model in RTL, but this was not completed before the synthesis deadline. As a result, the design has zero weight reuse: each weight is rewritten by the host CPU for every timestep evaluation.

## 9.2 AXI4-Lite Is Not Suitable as a Data Streaming Interface

AXI4-Lite was selected in M1 as an on-chip control bus. However, the implementation used it as the primary data path for weight streaming. AXI4-Lite requires a separate address phase and data phase for every transaction and does not support bursting. For a matrix multiply kernel that requires streaming  $784 \times 25 = 19,600$  weight values per inference, AXI4-Lite creates a massive overhead: 45 bus cycles per weight  $\times 19,600$  weights = 882,000 cycles of overhead for 19,600 cycles of useful computation (0.83% MAC utilization at peak,  $\sim 7.5\%$  with overlaps). AXI4-Stream or a DMA-fed FIFO would have been far more appropriate. This was identified as a risk in the M1 interface selection document but was not resolved in the implementation.

## 9.3 Single MAC Unit, Not Parallel PE Array

The M1 Heilmeier questionnaire described a systolic array approach. The implemented design has a single multiply-accumulate unit. A systolic array with even 16 PEs would have multiplied compute throughput by 16 $\times$ , and with shared weight registers would have significantly improved



MAC utilization. The decision to implement a single MAC was made at M2 to reduce scope and ensure the precision and interface designs were correct before scaling. The plan was to replicate the core at M3/M4, but this was not completed. In hindsight, the single-MAC design should have been described consistently throughout the milestones as a single-neuron prototype, not as an accelerator targeting 6,000 GFLOP/s throughput.

## 9.4 No Network-Level Accuracy Evaluation

The M2 precision document stated that network-level accuracy would be evaluated against an FP32 PyTorch baseline on MNIST and SHD datasets at M3. This evaluation was not completed. The precision error analysis (0.651% of threshold) provides a theoretical bound but does not confirm that a full SNN trained at FP32 precision produces equivalent classification accuracy when run with Q8.8 weights. This gap means the report's acceptability statement rests on a theoretical bound rather than an empirical accuracy measurement.

## 9.5 OpenLane 2 Power Report Limitations

The power estimate of 0.69 mW from OpenLane 2 at `nom_tt_025C_1v80` is based on switching activity estimates from synthesis. No gate-level simulation with real switching activity was run to capture dynamic power more accurately. The true power could differ by 2–3× depending on actual AXI4-Lite transaction rate and cache-miss behavior. This limitation is noted in the `power_report.txt`.

## 9.6 What Would Be Done Differently

If this project were restarted with the same schedule: (1) Implement an on-chip weight register file with at least 784 entries at M2, enabling single-load weight reuse at M3. (2) Replace AXI4-Lite with an AXI4-Stream input interface for weight streaming and a simple 32-bit status register interface for control. (3) Implement an 8-PE or 16-PE parallel compute core from M2 onward. (4) Evaluate network accuracy at M3 against PyTorch FP32 baseline. (5) Describe the accelerator throughout all milestones as a single-neuron prototype, not as a full network accelerator, to avoid misleading throughput projections.